
Introduzione agli Algoritmi

1.1 Cos'è un Algoritmo?

Definizione Una **procedura computazionale ben definita** che prende un valore (o un insieme di valori) come *input* e produce un valore (o un insieme di valori) come *output*.

Un algoritmo è quindi un insieme di operazioni non ambigue che trasformano un dato di ingresso nel risultato desiderato. Può essere paragonato, a livello concettuale, alle istruzioni contenute nel manuale di un set Lego o ai passaggi di una ricetta di cucina: una sequenza di passi meccanici eseguiti in un ordine specifico.

Un Esempio: L'Ordinamento

Uno dei problemi classici dell'informatica è l'ordinamento (o *sorting*). Consideriamo l'esempio di ordinare un mazzo di carte mescolato.

- **Input:** Un mazzo di carte non necessariamente ordinate (una sequenza di dati).
- **Output:** Una permutazione del mazzo fornito in input, in cui tutte le carte sono disposte in ordine crescente.

Ci serve una procedura meccanica per passare dall'input all'output.

Idea: Inserimento di una carta alla volta (Insertion Sort)

Possiamo dividere logicamente le carte in due gruppi: *Carte Ordinate* e *Carte Non Ordinate*. L'algoritmo procede nel modo seguente:

1. Introduzione agli Algoritmi

1. Inizialmente, la lista delle carte ordinate è vuota (non si hanno carte ordinate).
2. Finché rimangono carte non ordinate:
 - 1.1 Prendi una carta dalla lista di carte non ordinate e chiamala X .
 - 2.2 Metti X in fondo alla lista delle carte ordinate.
 - 3.3 **Finché** la carta che precede X ha valore maggiore del valore di X , **scambia** X con la carta che lo precede.

Nota Quando siamo riusciti ad ottenere un mazzo ordinato, per valutare la validità dell'algoritmo progettato, rimangono due domande fondamentali a cui rispondere:

- **Correttezza:** questo algoritmo funziona con ogni input possibile?
- **Efficienza:** quanti passi ci mettiamo ad ordinare un mazzo di n carte, al crescere di n ?

Analisi intuitiva dell'efficienza

Quante comparazioni dobbiamo fare nel caso peggiore? Inserire *una singola carta* nella lista delle carte ordinate richiede non più di n comparazioni (dove n è il numero di carte già in ordine). Siccome dobbiamo inserire n carte per ordinarle tutte, il numero totale di comparazioni è limitato superiormente da n^2 .

1.2 L'Importanza dell'Efficienza (Efficiency Matters)

L'efficienza diventa un concetto cruciale quando la dimensione dell'input cresce. Questo principio è evidente se analizziamo il problema della ricerca del **Massimo Comune Divisore** (MCD).

- **Input:** due numeri interi m ed n , con $m > n$.
- **Output:** il più grande intero che divide entrambi senza lasciare resto.

Approccio 1: Algoritmo Ingenuo

Una prima idea potrebbe essere quella di provare tutti i possibili divisori, partendo dal più grande al più piccolo.

```
1 def MCD_ingenuo(m, n):
2     mcd = 0
3     for i in range(1, math.floor(math.sqrt(n)) + 1):
4         if n % i == 0:
5             if m % (n//i) == 0:
6                 return n//i
```

```

7         elif m % i == 0:
8             mcd = i
9     return mcd

```

La **complessità** di questo algoritmo è problematica: nel caso peggiore fa iterazioni del ciclo proporzionali a $\sqrt{n}$. Se testiamo l'algoritmo con un input crittografico come $n = 2^{100}$, serviranno circa $\sim 2^{50}$ operazioni. Un tempo computazionalmente inaccettabile.

*Perché ci fermiamo a $\sqrt{n}$? L'intuizione matematica: I divisori di un numero viaggiano sempre in **coppie**.*

Prendiamo $n = 36$. Le coppie di numeri interi che moltiplicati danno 36 sono:

(1, 36), (2, 18), (3, 12), (4, 9), (6, 6)

Come si nota, in ogni coppia c'è sempre un numero "piccolo" e uno "grande". Il numero piccolo non supera mai la radice quadrata del numero stesso ($\sqrt{36} = 6$).

Se esistesse una coppia in cui entrambi i numeri sono maggiori di 6 (es. $7 \times 7 = 49$), il loro prodotto supererebbe 36, il che è impossibile!

*Quindi, per trovare **tutti** i divisori di n , basta cercare i numeri i da 1 fino a $\sqrt{n}$. Una volta trovato il numero piccolo i , il suo "compagno grande" si trova banalmente calcolando n/i .*

Approccio 2: L'Algoritmo di Euclide

Si basa su un'intuizione: *Se un numero divide sia m che n , allora deve dividere per forza anche la loro differenza e, di conseguenza, il loro resto.*

Esempio Pratico: Vogliamo trovare il MCD tra 1071 e 462.

1. Dividiamo il grande (1071) per il piccolo (462). Il quoziente è 2, ma a noi interessa il **resto**, che è 147. Euclide ci dice che $MCD(1071, 462)$ è uguale a $MCD(462, 147)$.
2. Ripetiamo: dividiamo 462 per 147. Il resto è 21. Ora il problema è $MCD(147, 21)$.
3. Ripetiamo: dividiamo 147 per 21. Il resto è **0**!
4. Quando il resto è zero, significa che l'ultimo divisore usato (**21**) è il nostro Massimo Comune Divisore.

Teorema : Proprietà di Euclide per il MCD

Data la divisione $m = q \cdot n + r$ (dove r è il resto):

- Se $r = 0$, allora $n \mid m$ e l'algoritmo termina: il MCD è n .
- Se $r \neq 0$, vale l'identità: $MCD(m, n) = MCD(n, r)$.

Tradotto in codice Python:

```
1 def Euclide(m, n):
2     while n != 0:           # Fino a quando il resto non e' zero...
3         vecchio_n = n
4         resto = m % n       # Calcolo il resto della divisione
5         m = vecchio_n      # Il vecchio divisore diventa il nuovo 'm'
6         n = resto          # Il resto diventa il nuovo divisore 'n'
7
8     return m
```

Perché Euclide è incredibilmente veloce?

L'algoritmo non sottrae semplicemente un numero dall'altro, ma prende i *resti*. Questo fa sì che i numeri collassino (si rimpiccioliscano) molto velocemente. Dimostriamolo matematicamente:

1. Partiamo dall'equazione della divisione: $m = q \cdot n + r$.
2. Poiché $m > n$, il quoziente q sarà come minimo 1 ($q \geq 1$).
3. Per le regole base della divisione, il divisore deve essere strettamente maggiore del resto: $n > r$.
4. Sostituiamo queste due verità ($q \geq 1$ e $n > r$) nell'equazione di partenza:

$$m = q \cdot n + r \geq 1 \cdot r + r = 2r$$

5. Ne deduciamo che $m \geq 2r$, ovvero girando la disequazione: $r \leq \frac{m}{2}$.

Risultato sulle prestazioni La formula $r \leq m/2$ ci dice che: **ogni due passaggi dell'algoritmo, il numero in input diventa meno della metà.**

Di conseguenza, il numero massimo di passi prima di arrivare a zero è logaritmico, precisamente limitato da $2 \log_2 m$.

Riprendendo l'esempio critico con $m = 2^{100}$:

- **Approccio ingenuo:** richiede circa 2^{50} cicli.
- **Euclide:** esegue al massimo $2 \cdot \log_2(2^{100}) = 2 \cdot 100 = 200$ passaggi. la formula $r \leq m/2$ ci dice che **ogni due passaggi dell'algoritmo, il numero in input diventa meno della metà.**

Per definizione matematica, il numero di volte in cui un valore può essere diviso per due è esattamente il suo **logaritmo in base 2** ($\log_2 m$).

Siccome questo dimezzamento avviene ogni 2 iterazioni, il numero totale massimo di passi sarà esattamente $2 \cdot \log_2 m$.